

**SIMATS ENGINEERING
ASSESSMENT TOOL 2**

COURSE CODE: CSA1402

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation (BL3)

Assessment Tool Weightage: 20%

QUESTIONS: 20

Total Marks:100

Annexure A

SHORT ANSWER TEST

Code of Conduct:

*I am Deepthi(192525343) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:Deepthi

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%				
Explanation	25%				
Application	20%				
Organization	15%				
Accuracy	10%				
Clarity	5%				

Faculty Incharge

Annexure A

SHORT ANSWER TEST

1. Construct language for the following Regular

Expression: $2(a|b)^*a(a|b)|(a|b)$

2. a) Illustrate the predictive parser for the following grammar. 4

$S \rightarrow (L) | a$

$L \rightarrow L, S |$

S

3. Analyse, is it possible, by modifying the grammar in any way to construct predictive parser for the language of

$S \rightarrow SS+ | SS^* | a$

String: "aa+a"

4

4. Eliminate left recursion from the grammar $S \rightarrow Sa|Sb|c$ 2
5. The following grammar contains common prefixes, making it unsuitable for predictive parsing.

$A \rightarrow abcD | abcE | f$

Apply left factoring to remove the common prefixes and rewrite the grammar.5

6. Consider the following statement:10

$Degree - F = Degree - C * 1.8 + 32$

Show the representation of this statement after each phase of the compiler and explain.

7. Consider the following C code fragment:4

```
sum = 0;
```

```
for (i = 1; i < 5; i++)  
    sum = sum + i;
```

Show the representation of the above code after each phase of the compiler.2

8. Consider the following grammar.

$S \rightarrow aABb, A \rightarrow c|\epsilon, B \rightarrow d|\epsilon$

Calculate FIRST and FOLLOW, Obtain the Predictive Parsing Table. not. Show the parser moves for the input string **acdb**.

9. Write a simple Lex program to recognize and count identifiers in a given input.2
10. Write the rule to Eliminate Left Recursion and Left Factoring.2
11. What are the actions performed by a shift-reduce parser? Explain each action with a suitable example.3
12. Consider the following grammar:

$S \rightarrow pBB \rightarrow qB|r$

Find the leftmost derivation and construct the parse tree for the input string pqqrr.3

13. Consider Grammar: $E \rightarrow E+T | T \quad T \rightarrow T^*F | F \quad F \rightarrow (E) | id$

Eliminate left recursion and find FIRST and FOLLOW 5

14. **For the following grammar, determine whether the string abcc is accepted using shift-reduce parsing. Show all shift and reduce operations.**

10

$$S \rightarrow ABA \rightarrow aA | aB \rightarrow cB | c$$

15. For the following grammar, verify whether the string **abbc** is valid using shift-reduce parsing.10

$$S \rightarrow aABA \rightarrow bA | bB \rightarrow c$$

Show the complete sequence of stack, input, and parser actions.

16. Consider the following grammar:10

$$S \rightarrow SS \rightarrow aABA \rightarrow bA | cB \rightarrow d$$

- Construct the canonical collection of LR(0) items.
- Construct the SLR(1) parsing table.

17. Consider the following grammar.

$$S \rightarrow iEtSS | a, S \rightarrow eS | \epsilon, E \rightarrow b$$

Obtain the Predictive Parsing Table. 5

18. Do left factoring in the following grammar: 3

$$A \rightarrow aAB | aA | a$$

$$B \rightarrow bB | b$$

19. Construct the LL(1) Parsing Table for the following grammars.7

$$S \rightarrow ABCA \rightarrow a | \epsilon B \rightarrow r | \epsilon C \rightarrow b | \epsilon$$

20. Construct the LL(1) Parsing Table for the following grammars.7

$$S \rightarrow 1S\# | qABCA \rightarrow a | bbDB \rightarrow a | \epsilon C \rightarrow b | \epsilon D \rightarrow c | \epsilon$$

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers

Step-4 stack implementation.

stack

input

output

\$S

$$3) S \rightarrow SS+ / SS* / a$$

string: "aa + a"

$$S \rightarrow aS'$$

$$S' \rightarrow +S' / *S' / \epsilon$$

$$S \rightarrow SS+$$

$$S \rightarrow SS*$$

$$S \rightarrow a$$

Stack implementation:-

stack	input	output
\$	aa + a \$	$S \rightarrow aS'$
\$ a S'	aa + a \$	
\$ S'	a + a \$	$S' \rightarrow \epsilon$
\$ S	a + a \$	$S \rightarrow aS'$
\$ S' a	a + a \$	
\$ S'	+ a \$	$S' \rightarrow \epsilon$
\$ S' +	+ a \$	$S' \rightarrow SS+S'$
\$ S'	a \$	
\$ S' a	a \$	$S' \rightarrow aS'$
\$ S' \$	\$	$S' \rightarrow \epsilon$

$$S \rightarrow Sa | Sb | c$$

left recursion

$$S \rightarrow cS'$$

$$S' \rightarrow aS' / bS' / \epsilon$$

2

$$A \rightarrow abcD / abcE / F$$

left factoring $A \rightarrow \alpha A / \alpha B$

$$A \rightarrow F$$

$$A' \rightarrow \underline{abcD} / \underline{abcE}$$

$$A' \rightarrow D / E / \epsilon$$

3

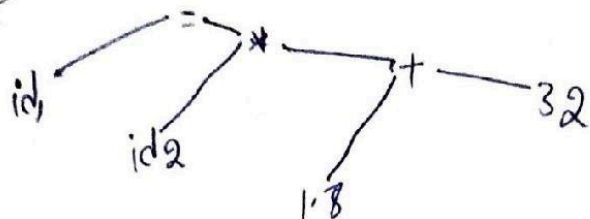
$$\text{Degree-F} = \text{Degree-C} * 1.8 + 32$$

Degree-F	id ₁	assigned
=	=	identified,
Degree-C	id ₂	operator
*	*	identifiers
1.8	1.8	operator
+	+	ident
32	32.0	operator
		ident.

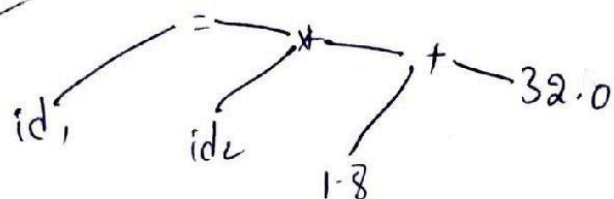
step-1 $cid_1 \Rightarrow cid_2 \times (1.8) + (32)$

$$id_1 = id_2 \times 1.8 + 32.$$

step-2



step-3



step-4

$$t_1 = 1.8 + 32.0$$

$$t_2 = id_2 \times t_1$$

$$t_3 = t_1 \times t_2$$

$$id_1 = t_3$$

step-5

$$t_1 = 1.8 + 32.0$$

$$t_2 = id_2 \times t_1$$

$$id_1 = t_2$$

step-6

$$\text{MOVF} = 32.0, \delta,$$

$$\text{ADDF} = 1.8, \delta,$$

$$\text{MULF} = id_2, \delta,$$

$$\text{MOVF} = \delta_1, \delta_2$$

$$\text{MOVF} = \delta_2.$$

Phases of compiler

Sum = Sum + ;

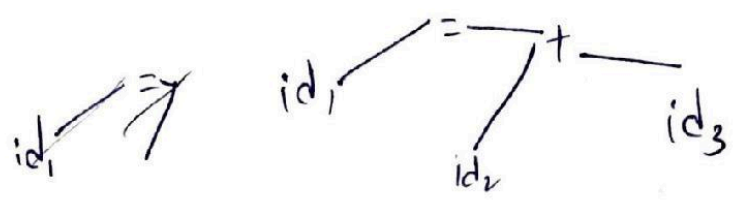
Sum	id,	identifier
=	=	operator
sum	id ₂	identifier
+	+	operator
i	id ₃	identifier

$\langle id_1 \rangle \langle = \rangle \langle id_2 \rangle \langle + \rangle \langle id_3 \rangle$

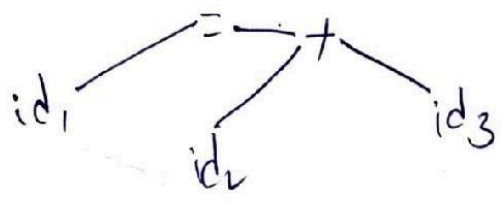
Step-1

$id_1 = id_2 + id_3$

Step-2



Step-3



Step-4

$t_1 = id_2 + id_3$
 $t_2 = t_1$
 $id_1 = t_2$

Step-5

$t_1 = id_2 + id_3$
 $id_1 = t_1$

47

Step-6

MOV F = id₃, r₁
Add F = id₂ +, r₁
MOV F = r₁, r₂
MOV F = r₂

$$S \rightarrow aABb.$$

$$A \rightarrow c/\epsilon$$

$$B \rightarrow d/\epsilon$$

$$\rightarrow \text{First}(S) = \{a\}$$

$$\text{First}(A) = \{c, \epsilon\}$$

$$\text{First}(B) = \{d, \epsilon\}$$

$$\rightarrow \text{Follow}(S) = \{\$, b\}$$

$$\text{Follow}(A) = \{\$, b\}$$

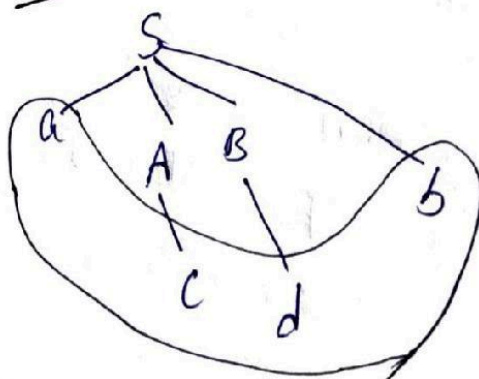
$$\text{Follow}(B) = \{\$, b\}$$

Predictive parsing Table:-

	a	b	c	d	\$
S	$S \rightarrow aABb$				
A		$A \rightarrow \epsilon$	$A \rightarrow c$		$A \rightarrow \epsilon$
B		$B \rightarrow \epsilon$		$B \rightarrow d$	$B \rightarrow \epsilon$

5

→ parsing tree if string:- acdb



Left factoring

$$(A \rightarrow \alpha A / \alpha B)$$

E.g $S \rightarrow icts / ictses / c$

$$A \rightarrow b$$

$\therefore$ Here in two productions same will be there eliminate that
Same thing in production

$$S \rightarrow cs'$$

$$s' \rightarrow es / \epsilon$$

$$A \rightarrow b$$

Left recursion

$$A \rightarrow A\alpha B$$

E.g $E \rightarrow E + T / T$

$\therefore$ It would be start with Same non-terminal.

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' \checkmark$$

$$\Rightarrow F \rightarrow (e)id$$

$\Rightarrow$ Then it will remain same. there is no ^{same} non-terminal.

$$F \rightarrow (e)id.$$

11)

for eg $S \rightarrow S+S$

$S \rightarrow S-S$

$S \rightarrow \gamma(S)$

$S \rightarrow a$

input string:- $a_1 a_2 a_3$

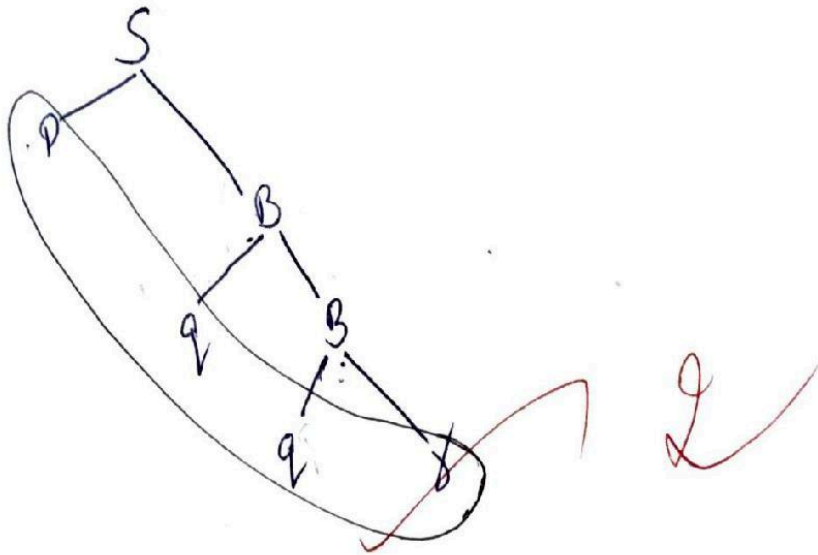
strings	input buffer	O/P parsing
$\$ B \$$	$a_1 a_2 a_3 \$$	Shift a_1 Action
$\$ a_1$	$(a_2 a_3) \$$	Shift a_2 Reduce $S \rightarrow a_1$
$\$ S$	$(a_2 a_3) \$$	Shift
$\$ S ($	$a_2 a_3 \$$	Shift a_2
$\$ S (a_2$	$-a_3) \$$	Reduce $S \rightarrow a_2$
$\$ S (S$	$-a_3) \$$	Shift
$\$ S (S-$	$a_3) \$$	Shift a_3
$\$ S (S-a_3$	$) \$$	Reduce $S \rightarrow a_3$
$\$ S (S-S$	$) \$$	Shift
$\$ S (S-S)$	$\$$	Shift
$\$ S (S)$	$\$$	Shift
$\$$	$\$$	Accept

2

$$S \rightarrow PB$$

$$B \rightarrow qB/\delta$$

input string: ppq δ



$E \rightarrow E + T / T$	$T \rightarrow T * F / F$	$F \rightarrow (E) / id$
$E \rightarrow TE'$	$T \rightarrow FT'$	$F \rightarrow (E) / id$
$E' \rightarrow +TE' / \epsilon$	$T \rightarrow *FT' / \epsilon$	

- 1) $E \rightarrow TE'$
- 2) $E' \rightarrow +TE' / \epsilon$
- 3) $T \rightarrow FT'$
- 4) $T' \rightarrow *FT' / \epsilon$
- 5) $F \rightarrow (E) / id$

First (e) $\rightarrow \{ (, id \}$

First (e') $\rightarrow \{ +, \epsilon \}$

First (τ) $\rightarrow \{ (, id \}$

First (τ') $\rightarrow \{ *, \epsilon \}$

First (f) $\rightarrow \{ (, id \}$

Follow (e) $\rightarrow \{ \$,) \}$

Follow (e') $\rightarrow \{ \$,) \}$

Follow (τ) $\rightarrow \{ \$, +, \$,) \}$

Follow (τ') $\rightarrow \{ +, \$,) \}$

Follow (f) $\rightarrow \{ *, +, \$,) \}$

Shift-reduce parsing

$S \rightarrow AB$

$A \rightarrow aA/a \rightarrow A \rightarrow a$ string abcc.

$B \rightarrow cB/c \rightarrow B \rightarrow c.$

string stack	input	output
\$	abcc\$	Action shift
\$a.	bcc\$	Reduce $A \rightarrow a$
\$A	bcc\$	shift
\$Ab	cc\$	

$\rightarrow aAB$
 $A \rightarrow bA/b$
 $B \rightarrow c$

IP: abbc.

stack	input	output
\$	abbc\$	Action, shift
\$a	bbc\$	shift
\$ab	bc\$	shift
\$abb	c\$	Reduce
\$abA	c\$	Reduce
\$aA	c\$	shift
\$aAa	\$	Reduce
\$aAAB	\$	Reduce.
\$s	\$	-

10

$s' \rightarrow s$

$s \rightarrow aAB$

$A \rightarrow bA/c$

$B \rightarrow d$

• SLR

• Canonical

$$I_0: S' \rightarrow S.$$

$$S \rightarrow \cdot aAB$$

$$A \rightarrow \cdot bA \mid \cdot c$$

$$B \rightarrow \cdot d$$

closure

$$\text{goto}(I_0, S) = I_1$$

$$S' \rightarrow S. \checkmark$$

$$\text{goto}(I_0, A) = I_2$$

$$S \rightarrow a \cdot AB$$

$$A \rightarrow b \cdot A$$

$$\text{goto}(I_0, B) = I_3$$

$$S \rightarrow a \cdot AB$$

$$\text{goto}(I_0, d) = I_4$$

$$B \rightarrow d. \checkmark$$

$$\text{goto}(I_0, c) = I_5$$

$$A \rightarrow c. \checkmark$$

$$\text{goto}(I_2, A) = I_6$$

$$A \rightarrow bA. \checkmark$$

$$S \rightarrow aA \cdot B$$

$$\text{goto}(I_6, B) = I_7$$

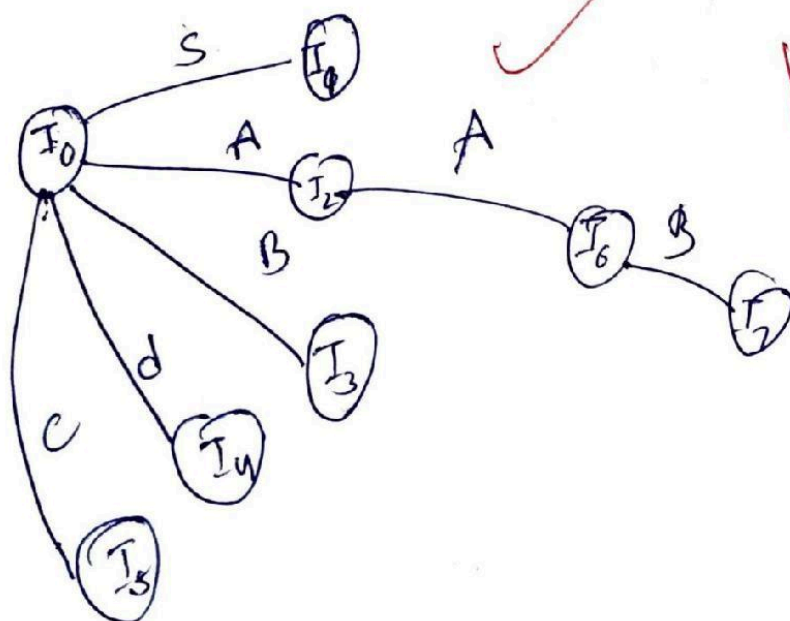
$$S \rightarrow aAB. \checkmark$$

$\text{first}(s') = \{a\}$
 $\text{first}(s) = \{a\}$
 $\text{first}(A) = \{b, c\}$
 $\text{first}(B) = \{d\}$

$\text{follow}(s') = \{\$ \}$
 $\text{follow}(s) = \{\$ \}$
 $\text{follow}(A) = \{d, \$ \}$
 $\text{follow}(B) = \{d, \$ \}$

state	closure					goto		
	a	b	c	d	\$	S	A	B
0			s_5	s_4		1	2	3
1					accept			
2						6		
3								
4				r_4	r_4			
5				r_3	r_3			
6				r_2	r_2			7
7					r_1			

$S \rightarrow aAB$
 $A \rightarrow bA$
 $A \rightarrow c$
 $B \rightarrow d$



$$S \rightarrow ietSS' / a \cdot$$

$$S' \rightarrow es / \epsilon$$

$$e \rightarrow b$$

$$\text{First}(S) = \{i, a\}$$

$$\text{First}(S') = \{e, \epsilon\}$$

$$\text{First}(e) = \{b\}$$

$$\text{Follow}(S) = \{\$, e\}$$

$$\text{Follow}(S') = \{b, e\}$$

$$\text{Follow}(e) = \{\$, e\}$$

Predictive table:-

	a	b	e	i	\$
S	$S \rightarrow iet$			$S \rightarrow a$	
S'			$S' \rightarrow es$		$S' \rightarrow \epsilon$
e		$e \rightarrow b$			

2

$$A \rightarrow aAB / aA / a$$

$$B \rightarrow bB / b$$

left factoring

$$A \rightarrow \alpha A / \alpha B$$

2

$$i) A \rightarrow a$$

$$A' \rightarrow B / \epsilon$$

$$B \rightarrow B$$

$S \rightarrow ABC$

$A \rightarrow a/\epsilon$

$B \rightarrow r/\epsilon$

$C \rightarrow b/\epsilon$

LL(1) Parsing table.

$First(S) = \{a, \epsilon\}$

$First(A) = \{a, \epsilon\}$

$First(B) = \{r, \epsilon\}$

$First(C) = \{b, \epsilon\}$

$Follow(S) = \{\$ \}$

$Follow(A) = \{\$ \}$

$Follow(B) = \{\$ \}$

$Follow(C) = \{\$ \}$

Predictive Parsing Table :

	a	b	r	\$
S	$S \rightarrow ABC$			$S \rightarrow \epsilon$
A	$A \rightarrow a$			$A \rightarrow \epsilon$
B			$B \rightarrow r$	$B \rightarrow \epsilon$
C		$C \rightarrow b$		$C \rightarrow \epsilon$

3

20) $S \rightarrow \epsilon S \# / qABC$

$A \rightarrow a / bbQ$

$B \rightarrow a / \epsilon$

$C \rightarrow b / \epsilon$

$Q \rightarrow c / \epsilon$

$\text{first}(S) = \{ \epsilon, q \}$

$\text{first}(A) = \{ a, b \}$

$\text{first}(B) = \{ a, \epsilon \}$

$\text{first}(C) = \{ b, \epsilon \}$

$\text{first}(Q) = \{ c, \epsilon \}$

$\text{Follow}(S) = \{ \#, \$ \}$

$\text{Follow}(A) = \{ \$, \# \}$

$\text{Follow}(B) = \{ \$, \# \}$

$\text{Follow}(C) = \{ \$, \# \}$

$\text{Follow}(Q) = \{ \$, \# \}$

Predictive Parsing table:-

	#	a	b	c	q	\$	
S					$S \rightarrow qABC$ $S \rightarrow \epsilon S \#$	$S \rightarrow \epsilon$	$S \rightarrow \epsilon S \#$
A		$A \rightarrow a$	$A \rightarrow bbQ$				
B		$B \rightarrow a$				$B \rightarrow \epsilon$	
C		$C \rightarrow \epsilon$	$C \rightarrow b$			$C \rightarrow \epsilon$	
Q		$Q \rightarrow \epsilon$		$Q \rightarrow c$		$Q \rightarrow \epsilon$	

3